

KOMPAS: KOOPMAN DYNAMICS MODELING AND PREDICTIVE CONTROL OF MULTI-TURN LLM BEHAVIOR

Anonymous authors

Paper under double-blind review

ABSTRACT

TODO(CP4)

1 INTRODUCTION

Multi-turn interaction turns a reply attribute of a frozen language model into a sequence with inertia. The attribute may be a word count, a sentence count, a readability level, a sentiment polarity, or whether a formatting constraint is met. What one reply does on such an attribute carries into the next reply, instead of being drawn fresh at every turn. Instruction following in particular is measured to decay over a dialogue, so a constraint met early is not met for free later. Deployments need to hold such an attribute at a target, such as a length budget or a reading level, across every turn. Holding a target requires steering, and inertia complicates steering. The same current reading can sit on a rising path or on a falling path, and the next reply differs between the two. A controller that reads one reply at a time cannot tell those two futures apart.

Prior literature partially addresses this control problem along three lines.

1) Latent-space controllers. Activation steering and its feedback variants, control barrier functions, and latent-space predictive control all act inside the network. They adjust activations, logits, or hidden states at every generated token. However, each of them needs white-box access to the target model and takes one token as its clock. The lever a deployment holds is one instruction per turn against a black-box API.

2) Prompt-level remedies. Work on multi-turn instability counters it at the prompt: a system prompt written once, a reminder on a fixed schedule, or a template fired by a per-turn detector. However, these remedies are stateless: none of them predicts what the next instruction will do to the next reply. The instruction is chosen from the current reply alone, and the two futures above receive the same instruction.

3) Control-theoretic views of language models. A third line writes prompting as a control problem and studies reachability, controllability, and steerability. Related work fits linear dynamics to token or embedding trajectories for analysis and for safety classification. However, these results are reachability statements, theoretical bounds, or detectors at the token level; none identifies a model that maps one instruction to the next reply’s attribute. What is missing is a dynamics model at the interaction level that uses only what a black-box caller can observe and carries the instruction as an explicit input.

The quantity a controller cares about is a property of a whole reply, not of any token inside it. A verifier scores the reply once it is complete, never partway through generation. The natural clock of the problem therefore ticks once per exchange. Some verifiers are programs that count words or check a format; others are rater models that score a level or a polarity. A black-box caller observes nothing else: the verifier’s score is the only signal returned, so the state of any model must be built from these scores. Controlling a room’s temperature needs no model of each air molecule. Controlling a reply’s attribute needs no model of each token, and no access to the hidden state that produced it.

Our Perspective: An Interaction-Level Koopman Perspective. We treat one prompt–response exchange as one step of a controlled dynamical system. The intuition is that the next reading decomposes into three parts: what recent history carries forward, what the current instruction pushes, and a constant offset. The state is the window of the last few readings, so a reading on a rising path

and the same reading on a falling path become two different states. A fixed prompting protocol turns a scalar command into the instruction text, which gives the instruction a coordinate the model can read. Linear predictors with inputs, fit on observables and paired with model predictive control, are an established route for nonlinear systems that must be planned over in real time. The same route fits this setting, since a window of verifier readings is a dictionary of observables and the command is the input. On this window we fit an affine one-step model by least squares on recorded exchanges. In its linear form this model is a delay regression with an exogenous input, and we do not claim otherwise. The operator view adds three things. The window is one choice of dictionary and a learned lifting is another; a horizon of steps has a closed form; predictive control applies unchanged. A lifting maps the window through a learned feature map and applies the same affine step there. We report the identity lifting throughout and keep a learned lifting as a comparison. The controller rolls every candidate command forward through this model, scores the predicted trajectory against the target, executes the best command, and replans after the next reply. Scoring a candidate costs a matrix product and a readout, with no extra query to the language model. Only the first command is executed, and the next selection starts from the observed reply rather than from a predicted one. **The key insight is that scoring on the predicted trajectory lets the controller send a command that differs from the target while the attribute is still moving.** A model that predicts well also selects well: the cost of the chosen command exceeds the best available cost by at most a term set by the prediction error. This yields **K**Oopman dynaMics modeling and **P**redictive control of **A**tttributes at the prompt **S**tep, or **KOMPAS**.

Our contributions are as follows.

1. We model the multi-turn dynamics of a reply attribute from verifier readings alone, with no access to the model’s hidden state. History beyond the current turn improves prediction on four of seven tracked attributes, and most of that gain is a per-item level rather than turn-to-turn dynamics.
2. We identify a command-conditioned Koopman model on delay coordinates of that reading, fit by least squares on recorded exchanges. Its linear instance ranks first on four of seven tracked attributes, and a learned lifting beats it on none of them.
3. We build a predictive controller that scores candidates inside the black box with no extra call to the target model, and we bound its selection error by the prediction error. It raises constraint satisfaction by 14.5 and 14.8 points over the uncontrolled Qwen3-4B and Qwen3-8B. It exceeds the stronger of two white-box latent controllers by 5.2 and 5.7 points on the same two models, and it ranks best in every model-by-subtask cell.

These control numbers are point estimates on three public instruction-following benchmarks, IF-Bench, IFEval, and COLLIE, with no interval reported.

2 PROBLEM FORMULATION

Two objectives share one interaction model: predicting the trajectory of a measured attribute over several exchanges, and driving that attribute to a target through prompt-level commands.

2.1 MODELING

At interaction step t , a frozen large language model receives a task input, the accumulated dialogue history, and a textual control instruction, and produces the next response. The resulting input–output interaction is

$$o_{t+1} \sim p_{\theta}(\cdot \mid x, \mathcal{H}_t, u_t), \quad y_{t+1} = V(o_{t+1}), \quad (1)$$

Here x denotes the task input, p_{θ} the frozen LLM, $\mathcal{H}_t$ the dialogue history available at step t , and u_t the textual instruction supplied by the controller. The response o_{t+1} is text, while V is a task-specific verifier that maps the completed response to a scalar attribute y_{t+1} , such as sentence length or sentiment polarity. We track one such attribute at a time, and a step corresponds to one complete prompt–response exchange. The verifier output is the only signal the controller observes, so the measured attribute y_t serves both for identification and for stating the objective.

Control decisions depend on the consequences of an instruction over several future turns. The model is therefore evaluated by its multi-step prediction error $\ell_h = (y_{t+h} - \hat{y}_{t+h|t})^2$ for $h = 1, \dots, H$.

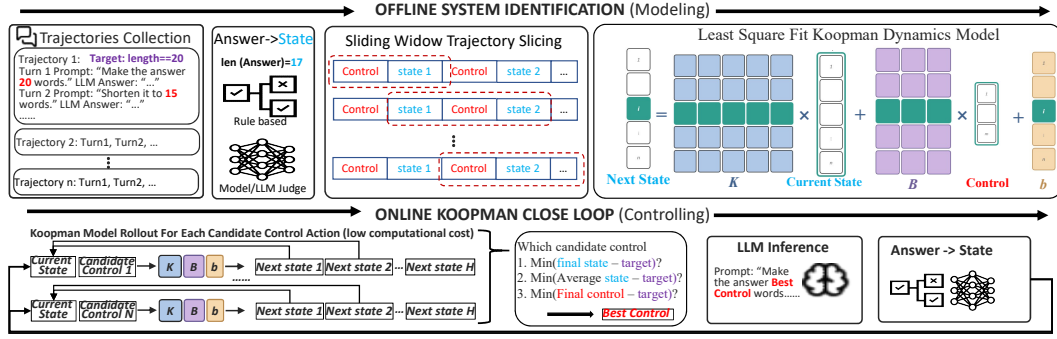


Figure 1: Overview of KOMPAS. Top, offline identification: recorded prompt–response trajectories are mapped by the task-specific measurement function V to attribute readings, a sliding window of readings forms the memory state s_t , and the affine Koopman dynamics model is fit by least squares. Bottom, online closed loop: a scalar command c_t is converted by a fixed prompting protocol into an instruction u_t for the frozen LLM; candidate commands are rolled forward through the learned dynamics, scored against the reference r , and the lowest-cost command is executed. The next response is measured and shifted into the state before the selection repeats.

Here $\hat{y}_{t+h|t}$ is the prediction of the attribute at step $t + h$ from information available at step t , and y_{t+h} is the realized measurement. The error should remain small throughout the horizon H , rather than only at the one-step prediction $h = 1$.

2.2 CONTROL

At each interaction step, the controller selects a scalar command c_t , which determines the textual instruction u_t applied to the LLM. Over a finite horizon of H steps, the control objective is

$$J_t = \sum_{h=1}^H (r - y_{t+h})^2. \quad (2)$$

Here $r \in \mathbb{R}$ denotes the desired value of the tracked attribute, and $e_t = r - y_t$ is the corresponding tracking error. Predictive control replaces the future measurements y_{t+h} , which are unavailable at decision time, by their predictions $\hat{y}_{t+h|t}$ when selecting the commands.

The controlled quantity is continuous, whereas the experiments report constraint satisfaction. The two views meet in the task-specific feasible set $\mathcal{Y}$: a response satisfies the constraint when $y_t \in \mathcal{Y}$, and the reference is chosen such that $r \in \mathcal{Y}$. Driving y_t toward r is therefore the mechanism through which the controller raises the rate of feasible responses.

The reference r specifies the desired output, whereas the command c_t is the action the controller encodes as the instruction u_t . A direct target-following controller sets $c_t = r$; predictive control allows $c_t \neq r$, which requires a state that retains enough interaction history for multi-step prediction.

3 METHOD

KOMPAS consists of an offline dynamics model and an online receding-horizon controller. Each completed response is mapped to the scalar attribute being controlled, recent attribute measurements form a finite-memory state, and a command-conditioned Koopman model predicts how that state evolves under a prompt instruction. At inference time, the controller rolls candidate commands forward through the learned dynamics, scores their predicted trajectories against the desired attribute, executes the best command, and replans after observing the next response. The frozen LLM p_θ is never updated, and candidate evaluation requires no additional query to it. Figure 1 summarizes this pipeline.

3.1 OBSERVABLE-STATE KOOPMAN DYNAMICS

The controller does not model text directly. Each completed response o_t is mapped by the task-specific measurement function V from Section 2.1 to a scalar attribute $y_t = V(o_t)$. The particular definition of V depends on the task and is given in the experimental setup; the dynamics model uses only the resulting scalar measurements.

A single measurement generally does not retain enough temporal context to predict the next one. The same value y_t , for example, may occur along an increasing or decreasing trajectory and therefore lead to different future responses. We represent this short-term history with delay coordinates,

$$s_t = [y_t \quad y_{t-1} \quad \cdots \quad y_{t-L}]^\top \in \mathbb{R}^{L+1}, \quad y_t = C s_t, \quad (3)$$

where $L \geq 1$ is the number of previous measurements retained and $C = [1, 0, \dots, 0]$ reads the current attribute from the state. We use s_t as a finite-memory approximation to the information in the dialogue history that is relevant for predicting the attribute; it is not intended to reconstruct the full textual state.

The controller acts through a scalar command rather than directly over the space of text strings. A fixed prompting protocol maps a command $c_t \in \mathcal{C} \subseteq \mathbb{R}$ to the textual instruction $u_t = \mathcal{P}(x, \mathcal{H}_t, c_t)$. Thus, after fixing $\mathcal{P}$, the controlled interaction can be viewed at the attribute level as a transition from (s_t, c_t) to s_{t+1} .

This controlled transition induces a command-conditioned Koopman operator on observables. For an observable f , we define $(\mathcal{K}_c f)(s) = \mathbb{E}[f(s_{t+1}) \mid s_t = s, c_t = c]$. The operator is linear in f even when the underlying prompt-response interaction is nonlinear. We use the delay coordinates in s_t as a finite dictionary of observables and approximate the induced dynamics with the affine model

$$\hat{s}_{t+1} = K s_t + B c_t + b, \quad \hat{y}_{t+1} = C \hat{s}_{t+1}. \quad (4)$$

Here $K \in \mathbb{R}^{(L+1) \times (L+1)}$ propagates the memory state, $B \in \mathbb{R}^{(L+1) \times 1}$ captures the effect of the command, and $b \in \mathbb{R}^{L+1}$ is a bias term. This parameterization deliberately shares one transition matrix across commands and lets the command enter through a single linear channel. These are modeling choices rather than consequences of Koopman linearity, and we do not assume that the frozen LLM admits an exact finite-dimensional Koopman representation.

More generally, Koopman dynamics may be defined on lifted coordinates $E_\phi(s_t)$ and decoded back to the measurement space. Our main model uses the identity lifting, $E_\phi(s_t) = s_t$, so the learned dynamics operate directly on the delay coordinates. We retain a learned lifting as an experimental comparison and give its training objective in Appendix A. The reported model is thus also a linear controlled regression on delay coordinates. The Koopman formulation places the identity and learned dictionaries in one observable-space framework and gives closed-form multi-step prediction.

3.2 OFFLINE SYSTEM IDENTIFICATION

We fit the dynamics model from recorded interaction trajectories. Sliding a window of $L + 1$ measurements over a trajectory produces transition tuples (s_t, c_t, s_{t+1}) , where c_t is the command applied before observing the successor state.

Because K , B , and b enter the one-step prediction linearly, the identity-lifting model is estimated by least squares,

$$(\hat{K}, \hat{B}, \hat{b}) = \arg \min_{K, B, b} \sum_t \|s_{t+1} - (K s_t + B c_t + b)\|_2^2. \quad (5)$$

The fit is performed entirely offline and does not modify the parameters θ of the target LLM. The range in which the learned dynamics can be trusted is consequently determined by the state-command combinations represented in the recorded trajectories. In particular, prediction under a new command history relies on generalization beyond the combinations observed during training.

3.3 RECEDING-HORIZON PROMPT CONTROL

At inference time, candidate commands are evaluated in the learned dynamics rather than by querying the LLM. Let $\mathcal{C}_t \subseteq \mathcal{C}$ be the finite nonempty set of candidate commands available at

step t . For each candidate $c \in \mathcal{C}_t$, we hold that command fixed only inside a hypothetical H -step rollout initialized from the observed state s_t . The predicted state obeys $\hat{s}_{t|t}(c) = s_t$ and $\hat{s}_{t+h|t}(c) = K\hat{s}_{t+h-1|t}(c) + Bc + b$, with $\hat{y}_{t+h|t}(c) = C\hat{s}_{t+h|t}(c)$.

Because the transition is affine, the rollout has the closed form $\hat{s}_{t+h|t}(c) = K^h s_t + \sum_{j=0}^{h-1} K^j (Bc + b)$, so candidate evaluation is linear propagation through the identified model and adds no call to p_θ .

Each candidate is scored by the deviation of its predicted trajectory from the desired attribute,

$$\hat{J}_t(c) = \sum_{h=1}^H w_h |\hat{y}_{t+h|t}(c) - r|^2, \quad c_t^* \in \arg \min_{c \in \mathcal{C}_t} \hat{J}_t(c), \quad w_h \geq 0. \quad (6)$$

Scoring the whole predicted trajectory allows the selected command to differ from the reference while the attribute is still moving toward it.

The controller executes only c_t^* : it sends $u_t = \mathcal{P}(x, \mathcal{H}_t, c_t^*)$, measures $y_{t+1} = V(o_{t+1})$ from the actual response, shifts it into the memory state, and repeats the selection from s_{t+1} . The held-command assumption applies only inside a candidate’s hypothetical rollout; the executed sequence is receding-horizon and may change command at every step.

3.4 WHY PREDICTION ACCURACY MATTERS

Predictive control is useful only when ranking candidates under the learned model approximately preserves their ranking under the true interaction. The following result makes this connection explicit for the held-command comparison performed at each control step.

For a candidate c , let y_h^c denote the realized attribute after h further interactions under the same held command, and let $J_t(c) = \sum_{h=1}^H w_h |y_h^c - r|^2$ be its realized trajectory cost. Suppose the one-step state residual along candidate rollouts is bounded by ε_{dyn} , the readout residual is bounded by ε_{rec} , and the readout is Lipschitz on the region visited by the rollouts. Then the resulting h -step attribute prediction error is bounded by some Δ_h , and the command selected by Eq. equation 6 satisfies

$$J_t(c_t^*) \leq \min_{c \in \mathcal{C}_t} J_t(c) + 2\delta_J, \quad (7)$$

where δ_J is the corresponding bound on the difference between predicted and realized candidate costs. Appendix F gives the explicit forms of Δ_h and δ_J and the proof.

The result is deliberately local: it concerns candidate selection at one replanning step and does not assert convergence of the full closed loop. Its role is to connect the modeling and control objectives: more accurate multi-step rollouts make the command selected from the candidate set closer to the best command that could have been chosen under the realized trajectory costs.

4 EXPERIMENTS

The experiments answer two questions, one per experiment line. **RQ1 (modeling)**. Does the learned operator predict a multi-turn attribute from observable readings alone? Its parts are (1a) history versus the current reading and (1b) the linear case versus a learned lifting or an LSTM (§4.2), and (1c) what the memory state carries (§4.3). **RQ2 (control)**. Does predictive control built on that operator raise constraint satisfaction on a frozen model? Its parts are (2a) against no control and (2b) against two white-box latent controllers, both in §4.5. §4.6 is an analysis rather than a further question: it states when a gain on RQ1 can become a gain on RQ2.

4.1 EXPERIMENTAL SETUP

Experiments cover two task groups. The first is self-built multi-turn tasks that form the modeling line. Table 1 carries seven of them: sentence length, Joint-3, Joint-2, sentiment, defense, constraint and CEFR. Joint-2 tracks two attributes of one sentence at once, word count and average word length; Joint-3 adds comma count. The other five tasks track a single attribute. Appendix B separately reports sentiment, average word length, defense and GSM8K-sharded for completeness. The second group is three public instruction-following benchmarks that form the control line: IFBench,

Table 1: Free-running prediction skill on seven multi-turn tasks. Higher is better. Best per column in bold, second underlined.

Predictor	Sentence len.	Joint-3	Joint-2	Sentiment	Defense	Constraint	CEFR
Stateless	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Last-turn	0.166	0.514	0.395	0.276	0.067	0.285	0.518
Memory w/o action	<u>0.764</u>	<u>0.824</u>	0.615	0.530	0.023	0.517	<u>0.561</u>
LSTM	<u>0.750</u>	<u>0.808</u>	<u>0.634</u>	0.423	-0.054	0.546	-0.101
KOMPAS (learned lifting)	0.752	0.821	0.627	0.396	0.021	0.471	-0.032
KOMPAS (linear)	0.769	0.826	0.640	<u>0.496</u>	<u>0.035</u>	<u>0.519</u>	0.567

IFEval and COLLIE. Together they carry eleven subtasks: WC, UWC, NS and NP for IFBench; KF, WC, NS and NP for IFEval; Word, Sent. and Para. for COLLIE. The modeling line runs on a single model, Qwen3-4B-Instruct-2507, which both produces and scores the self-built trajectories, so every number in Table 1 comes from that one model. The control line is separate: it reports two frozen target models, Qwen3-4B and Qwen3-8B, on runs supplied by a co-author.

The modeling line compares six predictors, the six rows of Table 1. Stateless is a memoryless null, Last-turn is a single-reading state, and Memory w/o action is a multi-turn state without the command term. LSTM is a recurrent sequence baseline, and KOMPAS (learned lifting) and KOMPAS (linear) are the two operator variants. KOMPAS (linear) sets the encoder and decoder to the identity map of §3.1, the reported instance throughout this paper. Window length, horizon, row counts, trajectory counts and the bootstrap protocol vary by task and are tabulated in Appendix C; every interval in §4.2 is a paired bootstrap over trajectories.

The control line compares four methods per frozen model, the four rows of Table 2: Base, TMPC, RE-Control and KOMPAS. Base applies no control, and its Access entry is ‘-’ in Table 2. TMPC and RE-Control are white-box latent controllers that require activation access to the frozen model. KOMPAS needs only the instruction it sends, marked black-box Access, since it never opens the model. TMPC and RE-Control are reproductions by a co-author rather than baselines the authors ran themselves.

A predictor that never beats guessing the best fixed trivial answer scores zero, and one that halves that guess’s error scores one half. We define the prediction skill score as $\text{Skill}_H = 1 - \sum_{h=1}^H \ell_h / \sum_{h=1}^H \ell_h^{\text{null}}$. Here ℓ_h is the squared forecast error at horizon step h from §2.1, and ℓ_h^{null} is the same quantity for the best trivial null. The best trivial null is the best-scoring memoryless baseline over the same rows. The score Skill_H is therefore the fraction of that null’s horizon error which the operator removes. A score of zero means tied with that null; a negative score means worse than it.

The control-line metric is the constraint-satisfaction score as reported for each benchmark, a per-subtask percentage with no interval attached. Its exact definition, what counts as a satisfied instruction per subtask, is not yet available from the colleague. The modeling line tests whether the operator predicts the trajectory; the control line tests whether acting on that prediction changes the outcome.

4.2 PREDICTIVE ACCURACY OF THE LEARNED OPERATOR

Question. RQ1a and RQ1b: whether multi-turn history improves prediction over the current reading alone, and whether a learned lifting or an LSTM beats the linear special case reported as ours. **Setup.** Setup, tasks and baselines follow §4.1; Table 1 reports the skill score defined there.

Findings, ranking and history. The linear special case, KOMPAS (linear), ranks first on four of seven columns and second on the other three. The driver is that all three second places are close: the linear case trails the top method by no more than 0.034 in point-estimate skill score. History beyond the current turn beats the current reading on four of seven columns, with paired bootstrap intervals excluding zero; the other three do not separate from zero. The driver is that the window exposes both level and trend, which a single reading cannot carry.

Findings, lifting and LSTM. A learned nonlinear lifting never beats the identity lifting with an interval excluding zero, on any of the seven columns; Figure 4(c) shows the per-column contrast

Table 2: Constraint satisfaction on IFBench, IFEval and COLLIE. Access states what each controller needs from the model. Values are point estimates; intervals are not yet available.

Model	Method	Access	IFBench				IFEval				COLLIE			Avg.
			WC	UWC	NS	NP	KF	WC	NS	NP	Word	Sent.	Para.	
Qwen3-4B	Base	–	25.5	7.0	36.7	12.4	20.7	44.6	33.9	28.9	23.0	51.3	11.9	26.9
	TMPC	white-box	35.2	9.9	46.3	19.0	29.1	53.4	43.1	37.5	33.2	59.6	21.3	35.2
	RE-Control	white-box	38.2	10.4	47.1	19.7	29.8	54.2	43.8	38.1	34.0	60.2	21.9	36.1
	KOMPAS	black-box	44.6	12.8	53.4	24.1	35.2	59.8	49.6	43.7	39.8	65.4	26.7	41.4
Qwen3-8B	Base	–	31.8	9.6	42.8	16.7	26.8	50.9	39.7	34.8	29.1	57.4	17.2	32.4
	TMPC	white-box	41.4	12.8	52.1	23.7	35.4	59.4	49.5	43.7	39.3	65.2	27.0	40.9
	RE-Control	white-box	42.6	13.2	52.9	24.2	36.0	60.1	50.2	44.3	40.0	65.9	27.6	41.5
	KOMPAS	black-box	50.8	15.9	59.7	29.3	41.8	65.7	55.8	49.9	46.1	71.4	32.8	47.2

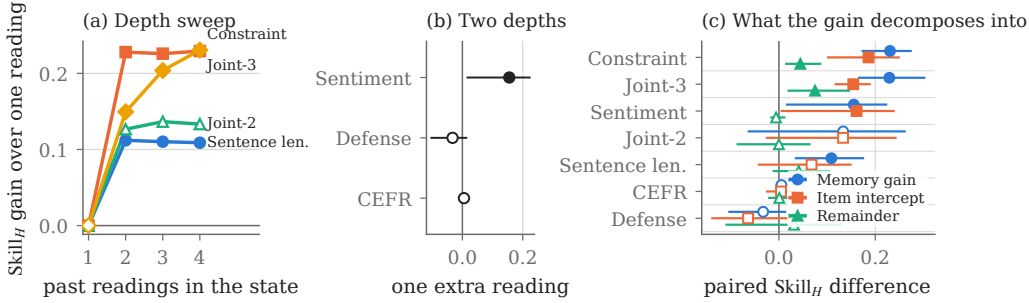


Figure 2: How deep the memory state has to be, and what its gain decomposes into, both under a matched training window. (a) Gain of a state holding d past readings over a state holding one, on the four tasks whose design allows a sweep. (b) The same gain for one extra reading on the three designs that carry two depths. (c) Per task of Table 1: the memory gain over Last-turn, the part a per-item intercept recovers, and the remainder beyond that intercept; intercepts are estimated on turns outside the scored window. In every panel a filled marker means the 95% paired bootstrap interval excludes zero and a hollow marker means it contains zero.

and §4.4 the reason. Against an LSTM sequence baseline, comparisons with intervals excluding zero split three ways: Joint-3 and CEFR favor the linear case, constraint favors the LSTM. No mechanism is established for the LSTM win, and none is proposed here.

Together, history beyond the current turn beats the current reading on four of seven tracked attributes, which answers RQ1a yes on those four. Neither a learned lifting nor an LSTM sequence model beats the reported linear special case on balance, which answers RQ1b no.

4.3 WHAT THE MEMORY STATE CARRIES

Question. RQ1c, in two parts: how many past readings the state needs, and whether the gain from memory is inter-turn dynamics or a fixed level per prompt item. **Setup.** Tasks, baselines and the skill score follow §4.1; both experiments vary one thing in the KOMPAS (linear) row of Table 1. Table 1 fits each row on every transition its state can be formed from. Both experiments here fit every depth on the same transitions, which separates the depth of the state from the size of its training set. Under this matched window the Table 1 memory gains shrink, with the two calibers compared in Appendix C.

Findings, depth. On three of the four sweepable tasks the second reading buys the whole gain, and the third and fourth readings add nothing that separates from zero (Figure 2a). Constraint is the exception: each added reading keeps paying, with every step separated from zero. Of the three two-depth designs, only sentiment gains from one extra reading (Figure 2b). The driver is that a level and a trend take two readings to express, and the attribute tasks carry little structure beyond that pair. No mechanism is established for why constraint keeps gaining, and none is proposed here.

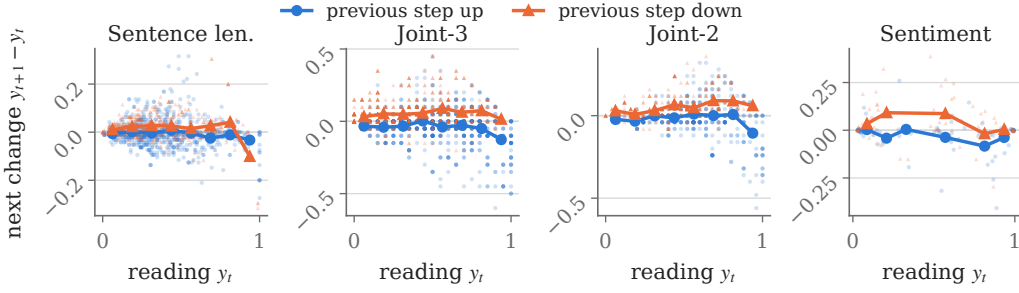


Figure 3: One reading is consistent with two opposite next steps. For each task, the next change against the current reading, split by the direction of the step before it; faint marks are transitions and each line is the mean of its group in eight equal-width bins. Transitions whose previous step was flat carry no direction and are dropped, which removes 7% of transitions on sentiment and up to 66% on Joint-2.

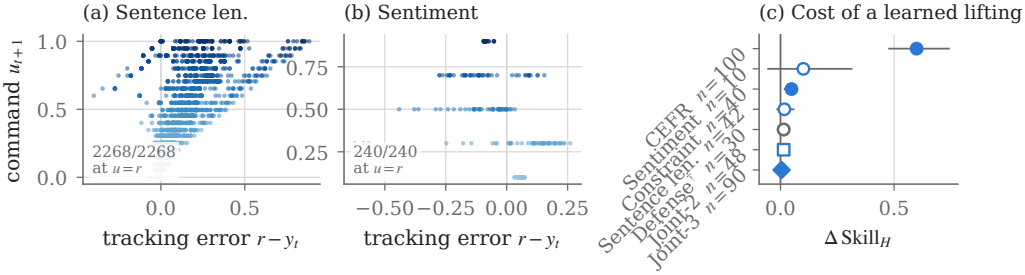


Figure 4: What the command channel carries, and what a learned lifting costs. (a) and (b) Applied command against tracking error, on the two Table 1 tasks that log a per-turn command; colour marks the target and repeats the vertical position, which is the point. (c) Paired Skill_H contrast of the linear case over the learned lifting, on all seven tasks, grouped by readout dimension: a circle marks the five one-dimensional readouts, a square the two-dimensional one and a diamond the three-dimensional one. A filled marker means the 95% interval excludes zero; the dagger marks a caveated entry.

Findings, item intercept. A per-item intercept added to the one-reading state recovers most of the memory gain: 81% on constraint and 67% on Joint-3. Only on those two tasks does the remainder beyond the intercept separate from zero; on sentiment and Joint-2 the intercept accounts for the whole gain (Figure 2c). The driver is that each prompt item holds its own level across turns, and several past readings estimate that level where one reading cannot.

Together, one extra reading carries most of what memory buys, and most of that is a per-item level rather than inter-turn dynamics; this answers RQ1c. A remainder attributable to dynamics is measured on two of seven tasks.

4.4 MECHANISMS READ OFF THE DATA

Question. Not a further RQ but a display: §4.2 argues two mechanisms from fitted scores, and this subsection shows each on the recorded readings. **Setup.** Figure 3 and Figure 4(a,b) read the released trajectory files and describe collected data, not model skill, so no value in them is comparable with a skill score. Figure 4(c) reads the paired contrasts of Table 1.

Findings, why a window helps. At a fixed reading the next change differs with the direction of the previous step on all four tasks (Figure 3). The separation does not rank the tasks the way the memory gain does: sentence length carries the largest gain in Table 1 and the smallest separation here. Read with §4.3, that ordering points at the per-item level rather than the turn-to-turn trend as the larger part of what the window buys.

Findings, what the command channel carries. In Figure 4(a,b) the applied command holds the value of the target across the full range of the tracking error, on every recorded transition of both tasks. This is the collection property behind the command-channel scope of §4.6 and Appendix G.

Findings, what a lifting has to find. The cost of a learned lifting concentrates on the one-dimensional readouts and is close to zero on the two multi-dimensional ones (Figure 4c). The cost does not fall with the number of trajectories: the largest sits on the task fit on 100 trajectories and the smallest on the task fit on 90. The driver is that a one-dimensional readout leaves a lifting little structure to expose, which is the part of the mechanism these seven columns support.

Together, the window separates two opposite next steps at one reading, and the cost of a lifting tracks the dimension of the readout rather than the amount of data.

4.5 CONSTRAINT SATISFACTION UNDER PREDICTIVE CONTROL

Question. RQ2a and RQ2b: whether predictive control built on the learned operator raises constraint satisfaction over the same frozen model with no control, and over the two white-box latent controllers. **Setup.** Setup, benchmarks and baselines follow §4.1; Table 2 reports the constraint-satisfaction score defined there.

Findings, gain over the uncontrolled baseline. Predictive control raises constraint satisfaction over the uncontrolled baseline by +14.5 points on Qwen3-4B and +14.8 on Qwen3-8B, averaged over eleven subtasks as point estimates with no seeds or intervals. The driver is that scoring happens on a candidate’s predicted trajectory, not its immediate effect. This lets the controller pick a command that differs from the target while the attribute is still moving.

Findings, gain over the white-box controllers. The same controller exceeds the stronger white-box baseline, RE-Control, by +5.2 points on Qwen3-4B and +5.7 on Qwen3-8B, again as point estimates with no interval. The access requirement is uneven, and it is uneven against us: RE-Control reads the model’s activations, while the predictive controller sees only text.

Taken together, predictive control raises constraint satisfaction over the uncontrolled baseline in all twenty-two model-by-subtask cells, a count rather than a significance claim, which answers RQ2a yes. It exceeds both white-box controllers in the same twenty-two cells at a weaker access requirement, which answers RQ2b yes.

4.6 FROM PREDICTION QUALITY TO CONTROL QUALITY

Question. Not a further RQ but the link between RQ1 and RQ2: how prediction quality relates to control quality, and where on the self-built tasks that link does not carry. **Setup.** Setup follows §4.1; the derivation is the selection bound of §3.4, not repeated here.

Findings, bound and cost. The selection bound of §3.4 keeps the cost of the selected command within twice the estimation error of the best candidate. It is theoretical, not measured, and it covers one held-command comparison at a single step rather than the replanned closed loop. Evaluating a candidate costs one linear propagation and a readout through the closed-form rollout of §3.3, adding no query to the target model. The driver is that the affine transition and the scalar command column collapse an h -step lookahead into a matrix power.

Cross-check, the equal-cost boundary. On the four self-built behavioral tasks, the closed loop ties the best equal-cost fixed schedule, with per-task numbers in Appendix D. On those four tasks the instruction’s effect depends little on when it is issued, which leaves planning almost nothing to exploit.

Cross-check, the command-channel scope. On four of the seven modeling columns the applied command is an exact affine function of the memory state, so the prediction design cannot estimate an independent action effect there. Appendix G reports the three columns where it can, all with intervals containing zero; this is a property of how the trajectories were collected, not a result about the controller.

Combined, the selection bound links prediction error to the quality of the command selected at a single step, while candidate evaluation adds no target-model queries. The equal-cost and

command-channel cross-checks delimit the conditions under which the predictive gains studied in RQ1 can translate into the control gains studied in RQ2.

5 CONCLUSION

TODO(CP4)

6 RELATED WORK

TODO(Phase 5)

A ESTIMATOR DETAILS AND THE LEARNED LIFTING

The reported instance is fit by the least-squares problem of Section 3.2, on the transition tuples $\mathcal{D} = \{(s_t, c_t, s_{t+1})\}$ constructed there. This appendix covers the learned-lifting variant instead, in which the encoder and the decoder are not the identity. What a lifting is, and why the reported instance leaves it off, is said in Section 3.1.

Training transition. Fitting the operator needs a one-step latent prediction from an encoded state and the applied command. Let

$$q_t = KE_\phi(s_t) + Bc_t + b. \quad (8)$$

Here q_t is the predicted encoding of s_{t+1} under the current parameters, computed from the encoder E_ϕ , the transition K , the command column B , and the bias b .

Training objective. Three losses jointly shape the encoder, decoder, and transition. We optimize

$$\begin{aligned} \mathcal{L}_{\text{rec}} &= \mathbb{E}_{\mathcal{D}} \left[\|s_t - D_\psi(E_\phi(s_t))\|_2^2 \right], \\ \mathcal{L}_{\text{lin}} &= \mathbb{E}_{\mathcal{D}} \left[\|E_\phi(s_{t+1}) - q_t\|_2^2 \right], \\ \mathcal{L}_{\text{pred}} &= \mathbb{E}_{\mathcal{D}} \left[\|s_{t+1} - D_\psi(q_t)\|_2^2 \right], \\ \mathcal{L} &= \lambda_{\text{rec}} \mathcal{L}_{\text{rec}} + \lambda_{\text{lin}} \mathcal{L}_{\text{lin}} + \lambda_{\text{pred}} \mathcal{L}_{\text{pred}}. \end{aligned} \quad (9)$$

The nonnegative weights λ_{rec} , λ_{lin} and λ_{pred} set how much each term contributes to the total. The reconstruction term $\mathcal{L}_{\text{rec}}$ preserves information present in the memory state s_t . The latent-consistency term $\mathcal{L}_{\text{lin}}$ pulls the encoded successor toward the affine prediction q_t . The prediction term $\mathcal{L}_{\text{pred}}$ ties that latent transition back to accuracy in the observed state space.

These three terms are complementary. Latent consistency alone admits uninformative constant encodings, since a constant encoder can drive $\mathcal{L}_{\text{lin}}$ to zero without capturing any structure. Reconstruction alone does not require the representation to support prediction, since $D_\psi(E_\phi(s_t))$ can recover s_t while carrying no information about s_{t+1} . Training updates (ϕ, ψ, K, B, b) , and the frozen language model’s parameters θ never change.

B SMALL-SAMPLE COLUMNS

TODO

C MODELING-LINE SETUP DETAILS AND MATCHED TRAINING WINDOW

Window length and horizon vary by task: window length ranges from 1 to 4 readings and horizon from 3 to 6 steps. Scored held-out rows per task range from 30 to 12,376, counted after windowing. The trajectories those rows come from number 40 to 100 for the seven columns of Table 1, and 4 to 30 for the columns of Appendix B. Every interval reported in §4.2 is a paired grouped bootstrap over trajectories, with 2000 resamples. Character length and formality are not shown in either table this round. A second backbone for the modeling line remains a planned arm with no submitted job.

The two experiment lines report different metrics in different reference frames. The modeling line’s skill score is referenced against a best-trivial-null baseline and can go negative, while the control line’s percentage carries no reference point.

The matched-window analysis of §4.3 fits every state depth on the same set of transitions; the per-task numbers behind Figure 2 are listed here. The caliber matters. On sentence length the memory gain over Last-turn is +0.603 under the default caliber of Table 1 and +0.109 under the matched window, so the two calibers answer different questions.

D EQUAL-COST CLOSED-LOOP COMPARISON

TODO

E ONE-STEP PREDICTION ERROR

TODO

F ASSUMPTIONS AND PROOF FOR THE SELECTION BOUND, EQ. EQUATION 7

The selection bound of Eq. equation 7 is stated on the memory state, which is the form the reported instance uses. The proof is written in the lifted coordinates of Appendix A, so that it covers the learned variant as well. Set $\xi_t = E_\phi(s_t)$ and write $g_\psi = C \circ D_\psi$ for the attribute readout. The identity lifting recovers the main-text statement, with $\xi_t = s_t$ and $g_\psi = C$.

For a candidate c , let s_h^c and y_h^c be the memory state and the attribute after h further interactions that hold c , starting from $s_0^c = s_t$, and set $\xi_h^c = E_\phi(s_h^c)$. The residuals of the fitted model along such a rollout are what the bound is stated in terms of,

$$\begin{aligned} \xi_{h+1}^c &= K\xi_h^c + Bc + b + \eta_h^c, & \|\eta_h^c\|_2 &\leq \varepsilon_{\text{dyn}}, \\ y_h^c &= g_\psi(\xi_h^c) + \nu_h^c, & |\nu_h^c| &\leq \varepsilon_{\text{rec}}. \end{aligned} \quad (10)$$

Here η_h^c is the one-step residual of the fitted transition and ν_h^c is the residual left in the readout. Both bounds are assumed to hold over the candidate rollouts, and both absorb imperfect finite-memory closure, model approximation and generation variability. Assume further that $\|K\|_2 \leq \kappa$, and that g_ψ is L_g -Lipschitz on the region the rollouts visit. For stochastic generation the statements hold on the event that these conditions are met.

The predicted and the actual rollout start from the same encoding, so the gap between them is the accumulated one-step residual carried through the readout,

$$|y_h^c - \widehat{y}_{t+h|t}(c)| \leq \varepsilon_{\text{rec}} + L_g \varepsilon_{\text{dyn}} \sum_{j=0}^{h-1} \kappa^j \triangleq \Delta_h. \quad (11)$$

Here Δ_h is the h -step attribute error bound of the selection bound of Eq. equation 7. It trades against the horizon: a longer horizon covers more distant consequences and asks the dynamics to hold over more steps. A contraction, $\kappa < 1$, would bound the geometric sum by $1/(1 - \kappa)$, and no contraction is assumed for the fitted model.

The cost of a candidate is a weighted sum of squared deviations, so a bound on each deviation gives a bound on the cost. Let R bound how far any attribute, predicted or actual, sits from the reference across candidates and rollout steps,

$$|J_t(c) - \widehat{J}_t(c)| \leq 2R \sum_{h=1}^H w_h \Delta_h \triangleq \delta_J. \quad (12)$$

Here δ_J bounds the gap between the predicted and the actual held-command cost of one candidate. The factor $2R$ comes from the identity $a^2 - b^2 = (a + b)(a - b)$ applied to each deviation pair.

Let $c_t^{\text{opt}} \in \arg \min_{c \in \mathcal{C}_t} J_t(c)$. Two applications of the cost bound around the definition of c_t^* close the argument,

$$\begin{aligned} J_t(c_t^*) &\leq \widehat{J}_t(c_t^*) + \delta_J \\ &\leq \widehat{J}_t(c_t^{\text{opt}}) + \delta_J \\ &\leq \min_{c \in \mathcal{C}_t} J_t(c) + 2\delta_J. \end{aligned} \tag{13}$$

The middle line holds because c_t^* minimizes the predicted cost. This is the second line of the selection bound of Eq. equation 7. It concerns the current held-command comparison, not the cumulative cost of the replanned loop, and it assumes nothing about the candidate set beyond its being finite and nonempty.

G COMMAND-CHANNEL SCOPE OF THE PREDICTION DESIGN

On four of the seven modeling columns of Table 1 the applied command equals the tracking error, an exact affine function of the memory state. The collection logs show it. On the two tasks that record a per-turn command, the applied command equals the target on every transition: 2268 of 2268 on sentence length, 240 of 240 on sentiment. The tracking error spans $[-0.40, +0.88]$ on sentence length across those same transitions. The two joint tasks record no per-turn scalar command, and their channel was checked by the rank test below. The prediction design therefore cannot estimate an independent action effect on those four columns. On the remaining three columns, defense, constraint and CEFR, the paired difference between the reported model and the memory-only variant is +0.0121, +0.0015 and +0.0061, with intervals containing zero. This is a property of how those trajectories were collected, not a result about the controller. The control experiments of §4.5 are the test of the command channel.

REFERENCES